



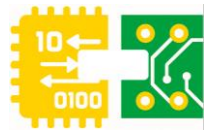
Hardware Engineering

Digital Technology

Lecture 09: Finite State Machines

Prof. Dr.-Ing. Ali Hayek

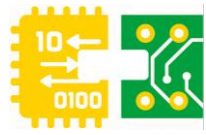
17.06.2026



- Content of **Digital Technology**: Chapter 1 to Chapter 11 [1]
 - Chapter 1) Introduction: Analog vs Digital and Number Systems
 - Chapter 2) Digital Circuitry and Interfacing
 - Chapter 3) Combinational Logic Design
 - Chapter 4) VHDL (Part 1): History and Introduction
 - Chapter 5) VHDL (Part 2): VHDL Modelling
 - Chapter 6) VHDL Simulation
 - Chapter 7) Sequential Logic Design
 - Chapter 8) VHDL (Part 3): Sequential and concurrent statements
 - **Chapter 9) VHDL (Part 4): State Machines**
 - Chapter 10) VHDL (Part 5): Libraries, Packages and subprograms
 - Chapter 11) Advanced Constructs

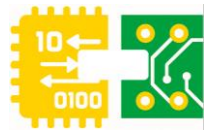
[1] B. J. Lameres, "Introduction to Logic Circuits & Logic Design with VHDL," Springer Publisher, 2017, 1st Edition

Recap:



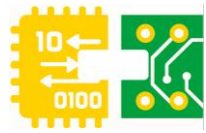
- Combinational circuits
- (Simple) Sequential circuits

- What about more complex sequential circuits?
- For example: Finite State Machines!



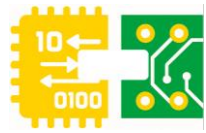
What are Finite state machines and why do we need them?
Which different types of finite state machines do you know?
How to design FSMs in VHDL?

Finite-State Machines: Intro

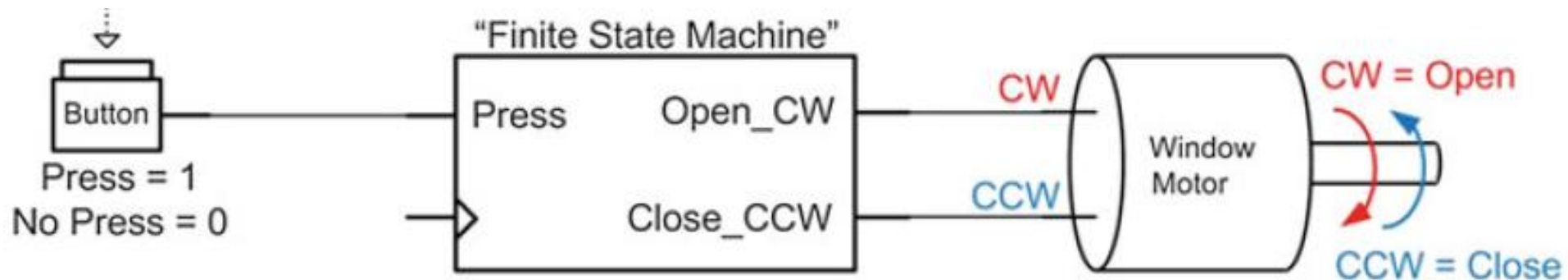


- Finite-state machine (FSM): circuit with a predefined number of states (i.e., a finite number of states)
 - Only one state machine allowed at time
 - Circuit transitions between states based on a triggering event, most commonly the edge of a clock
 - Circuit decision on the next state to transition to based on a history of past states and through the use of states and a predefined sequence of transitions
 - More intelligent outputs in comparison with a simple combinational logic circuit, whose outputs based only on the current values of the inputs
 - Different types depending on the application

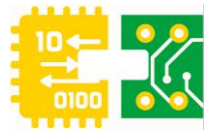
Finite-State Machines: Intro



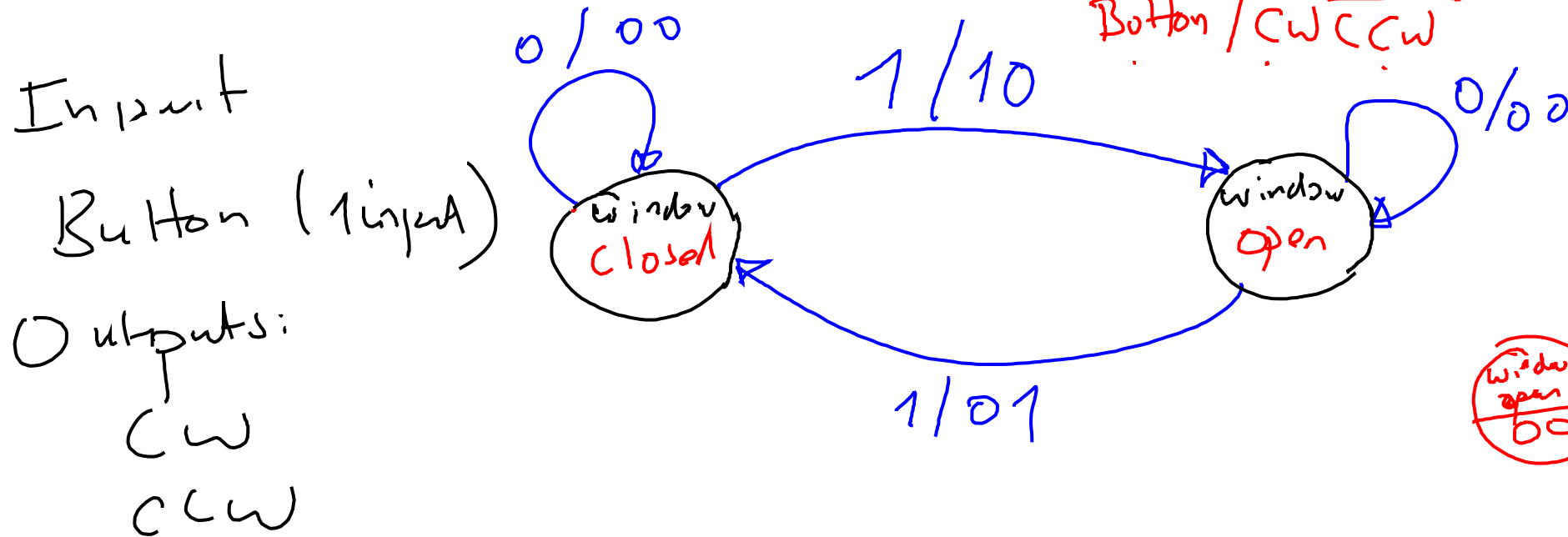
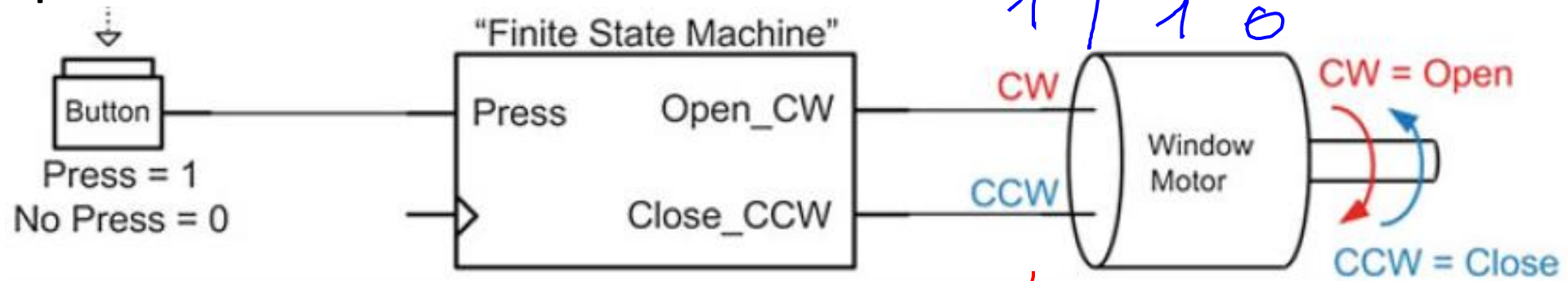
- Example: word description of a push-button window controller
 - System design to open and close a window with the push of a button
 - Motor with two inputs: clockwise (CW) for the motor spinning to open the window and counterclockwise (CCW) for the motor spinning to close the window
 - Motor movement only until the window achieves the correct position. User can press only open or closed position

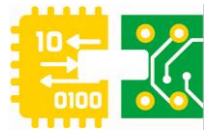


Finite-State Machines: Intro



- Example:





Inputs

Press

0

0

1

1

1

Current state

Q

0

1

0

1

1

Next state

Q'

0

1

1

0

↓

$Q \oplus \text{Press}$

Outputs

cw

ccw

0

0

0

0

1

0

0

1

↓

$\text{Press} \cdot \overline{Q}$

↓

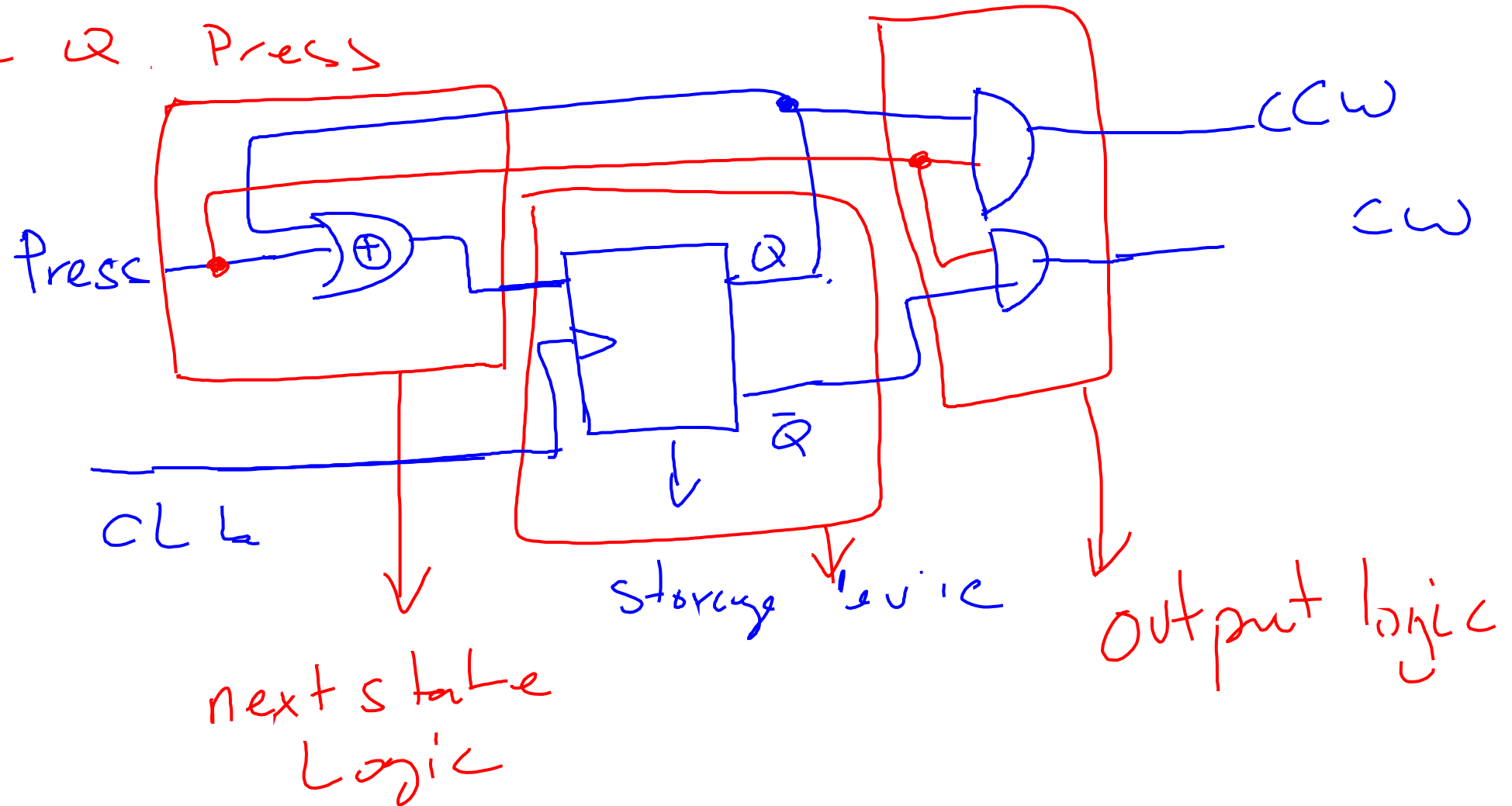
$\text{Press} \cdot Q$

Open $\Rightarrow 1$
close $\Rightarrow 0$

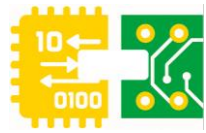
$$Q' = Q \oplus \text{Press}$$

$$CW = \overline{Q} \cdot \text{Press}$$

$$CCW = Q \cdot \text{Press}$$

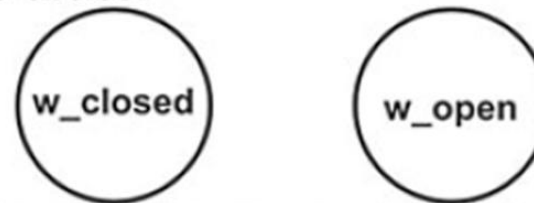


Finite-State Machines: Intro

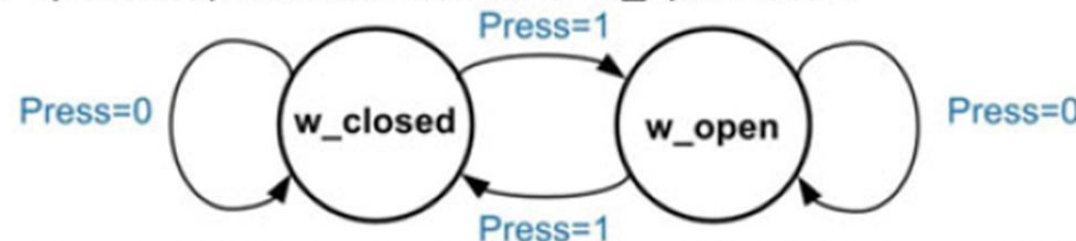


- Application of state diagram on the push-button window controller

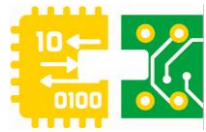
1) Defining the States - For this design, we will define two finite states. The first state is when the window is in the closed position. Let's call this state "w_closed". The second state is when the window is in the open position. Let's call this state "w_open". Each of these two states will be represented in the state diagram as circles. The names of the states are written inside of the circles.



2) Defining the Transitions - We now describe the transitions between states using arrows and labeling the arrows with the input conditions that trigger each transition. For this design, when the machine is in the "w_closed" state, a button press (Press=1) will cause a transition to the "w_open" state. When the button is not pressed, the machine will remain in the "w_closed" state (Press=0). When the machine is in the "w_open" state, a button press (Press=1) will cause a transition to the "w_closed" state, while the button not being pressed (Press=0) will keep the machine in the "w_open" state.

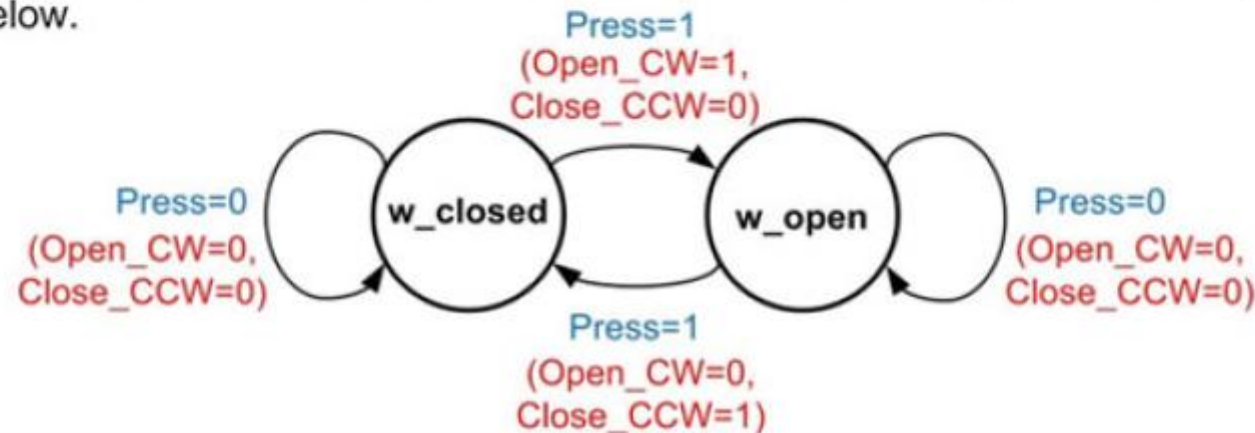


Finite-State Machines: Intro

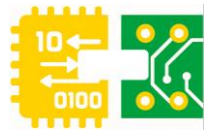


- Application of **state diagram** on the push-button window controller

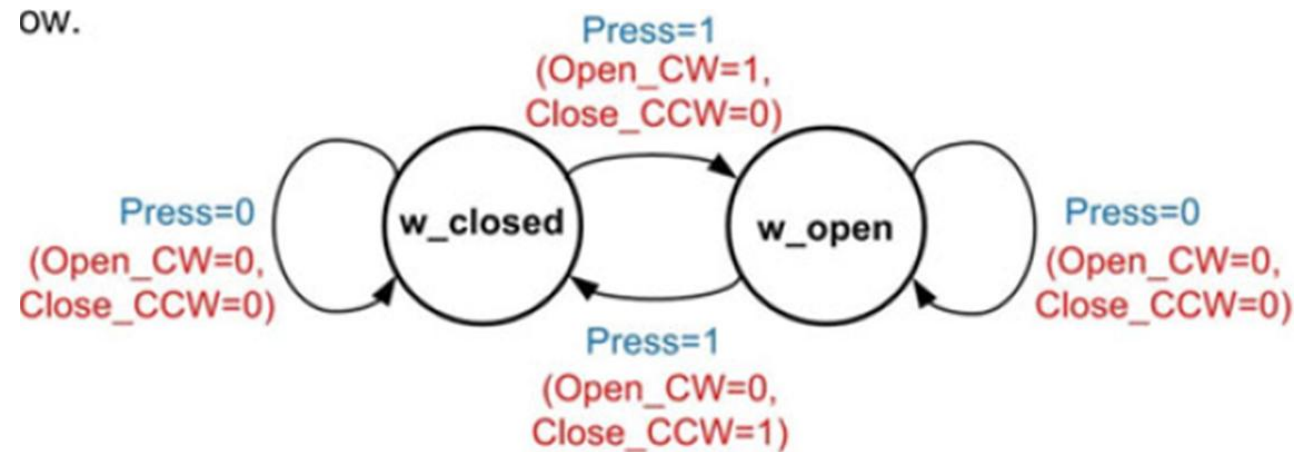
3) Defining the Outputs – We now describe the outputs of the system. For this design, the system will output the appropriate motor control signals upon a button press. This means that the outputs depend on both the current state and the current inputs. This is by definition a *Mealy Machine*. As such, the outputs are listed next to the state transitions. By listing the outputs in this location, both the current state and the input values producing the outputs are indicated. When this machine is in either the `w_closed` or `w_open` states and the button is NOT pressed, the outputs `Open_CW` and `Close_CCW` are both 0's. When the machine is in `w_closed` state and the button is pressed, the `Open_CW` output is asserted to rotate the motor clockwise and open the window. When the machine is in `w_open` state and the button is pressed, the `Close_CCW` output is asserted to rotate the motor counterclockwise and close the window. The final state diagram for this system is shown below.



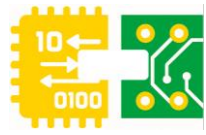
Finite-State Machines: Intro



- State Transition Tables
 - Representation of the state diagram as a truth table



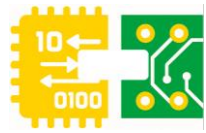
(Input)			(Outputs)	
Current State	Press	Next State	Open_CW	Close_CCW
w_closed	0	w_closed	0	0
w_closed	1	w_open	1	0
w_open	0	w_open	0	0
w_open	1	w_closed	0	1



How FSMs circuits are built?

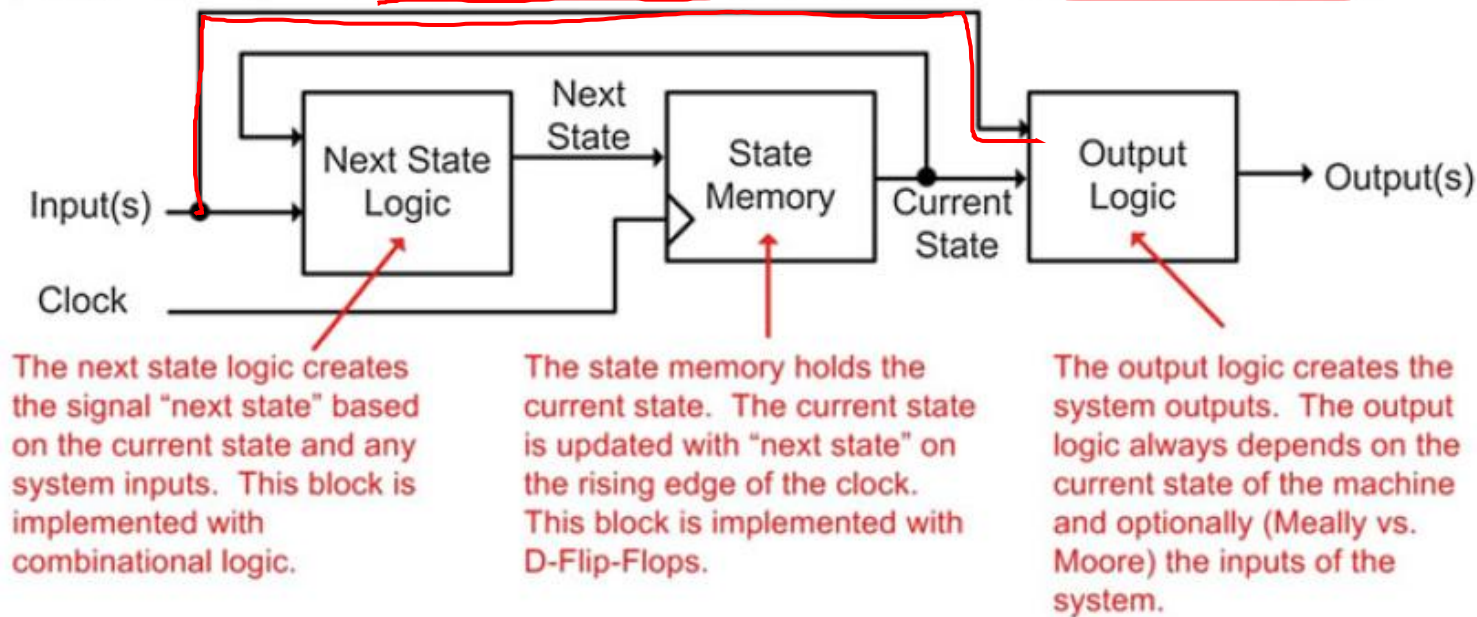
Which different types of finite state machines do you know?

Finite-State Machines: Synthesis



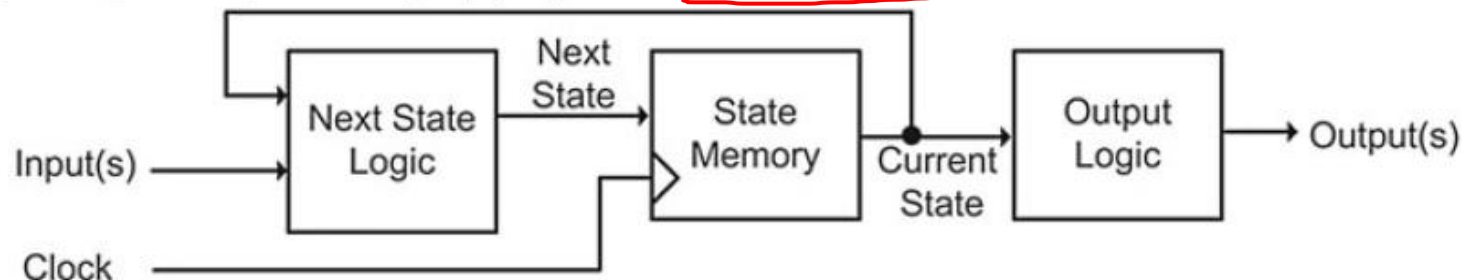
- Main components of a finite-state machine

Mealy Machine – The output(s) depend on both the current state and system input(s).

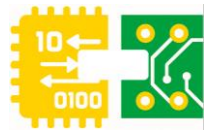


Which one is better?

Moore Machine – The output(s) depends only on the current state.



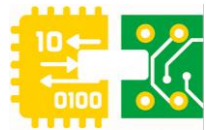
Finite-State Machines: Synthesis



- Moore and Mealy FSMs are functionally equivalent.
 - Equivalent Mealy FSM can be derived from Moore FSM and vice versa.
- Mealy FSM has richer description and usually requires less number of states
 - Smaller circuit area.
- Mealy FSM computes outputs as soon as inputs change.
 - Mealy FSM responds one clock cycle sooner than equivalent Moore FSM.
 - There are direct paths from inputs to outputs – can cause output glitches.
- Moore FSM has no combinational path between inputs and outputs.
 - Less likely to affect the critical path of the entire circuit

Choose a Mealy machine if response time and hardware efficiency are your top priorities. Choose a Moore machine if you need predictable, glitch-free outputs and want simpler timing analysis

Finite-State Machines: Synthesis



- How do we know how many FFs are required?

decimal: $2^3 = 8$

(41)

→ 21

41

6

⇒ 8 states ⇒

3 FFs ✓

8 states ⇒ 8 FFs

One Hot Coding

0000 000 1
0000 001 0
0000 010 0
0000 100 0

⋮

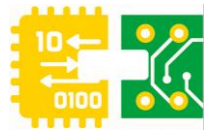
1000 0000

(3) Gray coding:

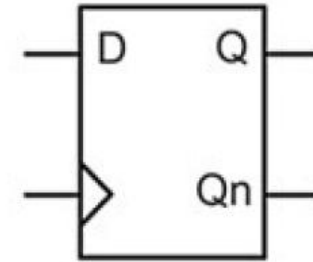
0 0 0
0 0 1
0 1 1

⋮

Finite-State Machines: Synthesis



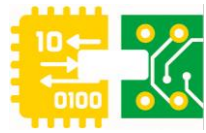
- State Memory: composed of one or more D-flip-flops and holds the current state of the machine
 - Number of D-flip-flops required as a function of the encoded states
- Example of binary encoding
 - n D-flip-flops: 2^n encoded states



Clk	D	Q	Qn	
0	X	Last Q	Last Qn	Store
1	X	Last Q	Last Qn	Store
$\downarrow$	0	0	1	Update
$\downarrow$	1	1	0	Update

Problem: You are designing a state machine that has 41 unique states and are going to encode the states in binary. How many D-Flip-Flops do you need?

Finite-State Machines: Synthesis



Problem: You are designing a state machine that has 41 unique states and are going to encode the states in binary. How many D-Flip-Flops do you need?

Solution: Each D-Flip-Flops will hold one bit of the state code. If the state memory has n -bits, it can encode 2^n states using binary encoding. We can use logarithms in order to solve for the n in the exponent.

$$2^n = (\text{\# of states})$$

$$\log(2^n) = \log(\text{\# of states})$$

$$n \cdot \log(2) = \log(\text{\# of states})$$

$$n = \frac{\log(\text{\# of states})}{\log(2)}$$

$$n = \frac{\log(41)}{\log(2)}$$

$$n = 5.36$$

Decimal

0 0

0 1

1 0

1 1

Gray Code

0 0

0 1

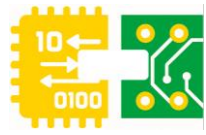
1 1

1 0

Rounding up to the next whole number means that we need 6 bits, or 6-D-Flip-Flops to encode 41 states in binary.

Check: To check this, let's plug 6 back into the original expression. If we have 6 bits, we can encode 2^6 states, or 64 states. This is enough to encode our 41 states. If we had 1 less bit (e.g., 5), we could only encode up to $2^5=32$ states, so we require 6 bits for this state encoding. Note that not all of the possible binary values are used as state codes.

Finite-State Machines: Synthesis



- Gray code encoding
 - Reduce of the number of bit transitions on the state codes in case of a linear sequence transition
 - Reduce of the number of bit transitions implying into less amount of power consumption and noise generated by the circuits
- Example of n-bit gray code pattern

Creating an n-bit Gray Code Pattern

A gray code sequence begins with the known 2-bit pattern of 00, 01, 11, 10.

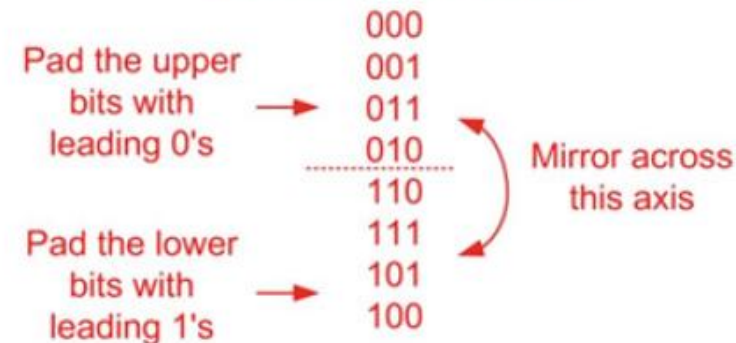
In order to increase the number of bits, the existing pattern is mirrored across an imaginary horizontal axis below the existing pattern. The bits above the axis are padded with leading 0's, and the bits below the axis are padded with leading 1's. This turns a 2-bit gray code pattern into a 3-bit pattern preserving the characteristic that each code only differs by its neighbor by one bit.

This process is repeated to create a 4-bit gray code pattern.

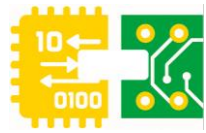
2-bit Gray Code Pattern

00
01
11
10

3-bit Gray Code Pattern



Finite-State Machines: Synthesis



- One-hot encoding
 - A separate D-flip-flop asserted for each state in the machine
 - Less chance of entering into an inadvertent state due to the different propagation delays

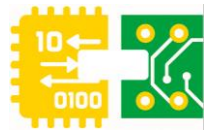
A state machine has eight unique states named S0, S1, ... S7. The following is an example of how these states can be encoded using binary, gray code and one-hot.

<u>State Name</u>	<u>Binary</u>	<u>Gray Code</u>	<u>One-Hot</u>
S0	000	000	00000001
S1	001	001	00000010
S2	010	011	00000100
S3	011	010	00001000
S4	100	110	00010000
S5	101	111	00100000
S6	110	101	01000000
S7	111	100	10000000

Handwritten red annotations:

- An arrow points from the word "error" to the S2 Binary value (010).
- Below the arrow, the text "001" is written.
- Another arrow points from the word "error" to the S0 One-Hot value (00000001).
- To the right of this arrow, the text "00000011" is written.

Finite-State Machines: Synthesis



- One-hot encoding
 - A separate D-flip-flop asserted for each state in the machine
 - Less chance of entering into an inadvertent state due to the different propagation delays
- Application of state encoding on the push-button window controller

This state machine contains two states, `w_closed` and `w_open`. The following are the three possible ways these states could be encoded.

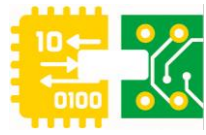
State Name	Binary	Gray Code	One-Hot
<code>w_closed</code>	0	0	01
<code>w_open</code>	1	1	10

Since this machine is so small, there is no difference between the binary and gray code approaches. Both of these techniques will require one D-Flip-Flop to hold the state code. The one-hot approach will require two D-Flip-Flops. Let's choose binary state encoding for this example. Let's use the state variable names `Q_cur` and `Q_nxt`.

Once the state codes and state variables are chosen, the state transition table is updated with the new detailed information about the design.

Current State		Input		Next State		Outputs	
	<code>Q_cur</code>		<code>Press</code>		<code>Q_nxt</code>	<code>Open_CW</code>	<code>Close_CCW</code>
<code>w_closed</code>	0		0	<code>w_closed</code>	0	0	0
<code>w_closed</code>	0		1	<code>w_open</code>	1	1	0
<code>w_open</code>	1		0	<code>w_open</code>	1	0	0
<code>w_open</code>	1		1	<code>w_closed</code>	0	0	1

Finite-State Machines: Synthesis



- Next state logic
 - Computation of the values of the next state variables based on the current state and the system inputs

We need to synthesize the combinational logic circuit that will create the next state logic for Q_{nxt} . The behavior of this combinational logic circuit is described in the state transition table. In order to visualize where this information is within the table, let's pull it out and put it into a traditional truth table format.

Current State		Input		Next State		Outputs	
	Q_{cur}		Press		Q_{nxt}	Open_CW	Close_CCW
w_closed	0	0	0	w_closed	0	0	0
w_closed	0	0	1	w_open	1	1	0
w_open	1	0	0	w_open	1	0	0
w_open	1	0	1	w_closed	0	0	1

↑ ↑
These columns are the inputs to the next state logic.

↑
This column is the desired output for the next state logic variable Q_{nxt} .

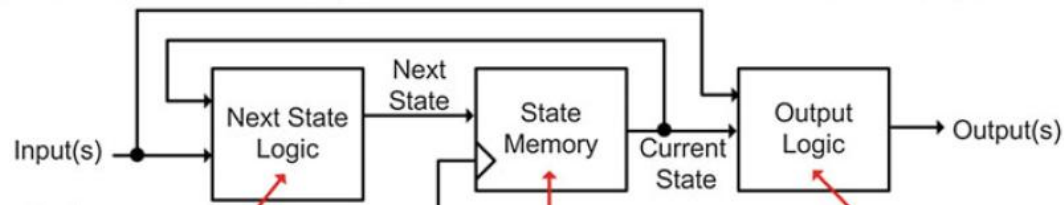
Q_{cur}	Press	Q_{nxt}
0	0	0
0	1	1
1	0	1
1	1	0

Press	Q_{cur}	
	0	1
0	0	1
1	1	0

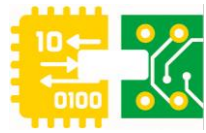
$$Q_{nxt} = (Q_{cur}' \cdot Press) + (Q_{cur} \cdot Press')$$

or

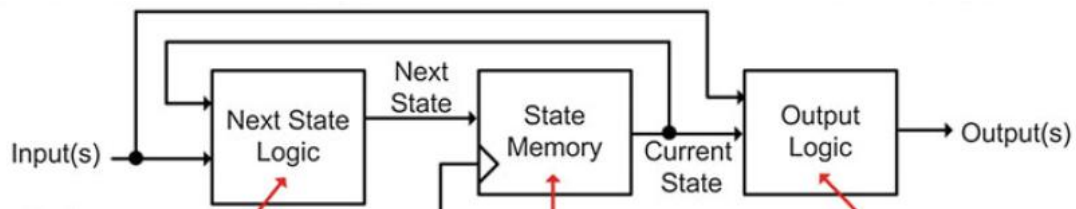
$$Q_{nxt} = Q_{cur} \oplus Press$$



Finite-State Machines: Synthesis



- Output logic
 - Compute the values of the system outputs based on the current state and, in the case of a Mealy machine, the system inputs
 - Each of the output signals with a dedicated combinational logic circuit



We need to synthesize the combinational logic circuits that will create the output logic for the signals "Open_CW" and "Close_CCW". The behavior of this combinational logic circuit is described in the state transition table. Again, in order to visualize where this information is within the table, let's pull it out and put it into traditional truth table formats.

Current State		Input		Next State		Outputs	
	Q_cur		Press		Q_nxt	Open_CW	Close_CCW
w_closed	0		0	w_closed	0	0	0
w_closed	0		1	w_open	1	1	0
w_open	1		0	w_open	1	0	0
w_open	1		1	w_closed	0	0	1

↑ ↑
These columns are the inputs to the output logic.

↑ ↑
These columns are the desired behavior of the outputs.

Q_cur	Press	Open_CW
0	0	0
0	1	1
1	0	0
1	1	0

		Q_cur	
		0	1
Press	0	0	0
	1	1	0

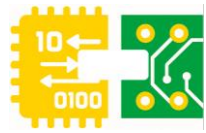
$$\text{Open_CW} = Q_cur' \cdot \text{Press}$$

Q_cur	Press	Close_CCW
0	0	0
0	1	0
1	0	0
1	1	1

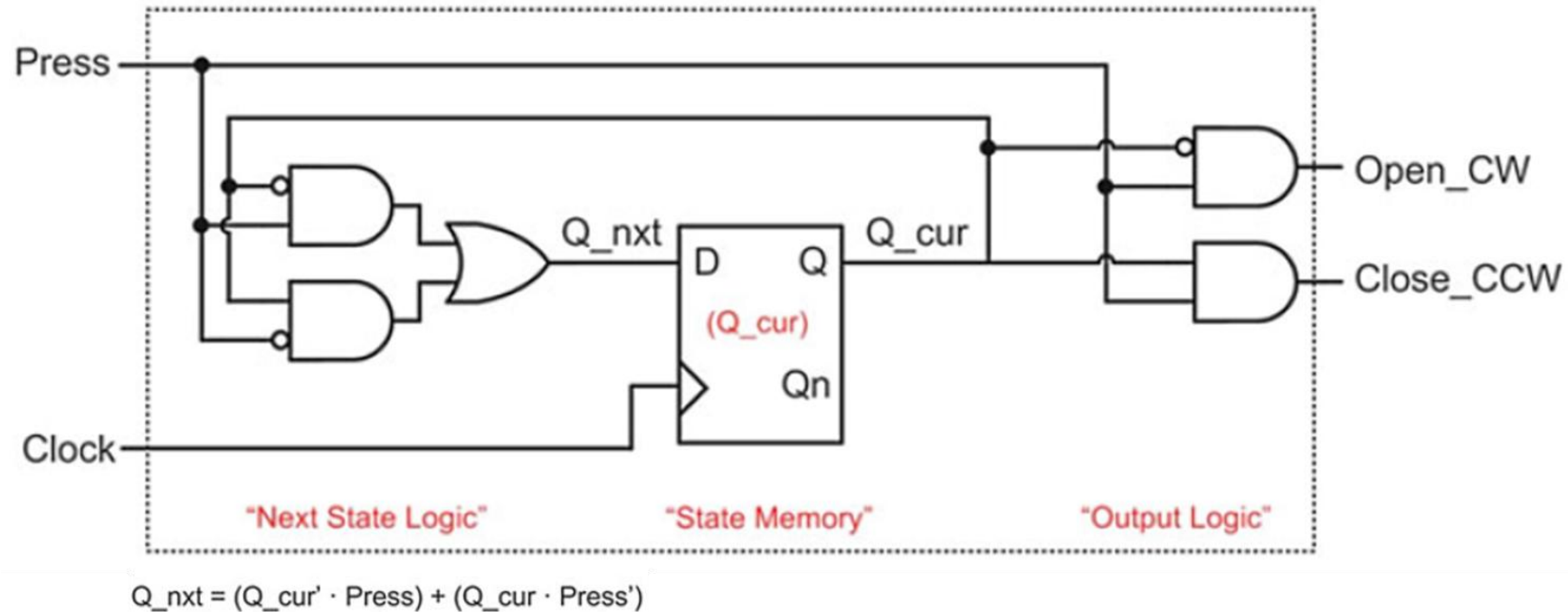
		Q_cur	
		0	1
Press	0	0	0
	1	0	1

$$\text{Close_CCW} = Q_cur \cdot \text{Press}$$

Finite-State Machines: Synthesis

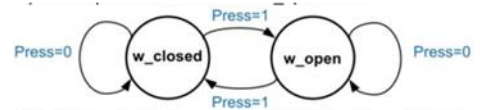
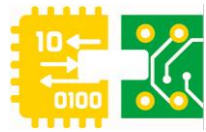


- Final logic diagram

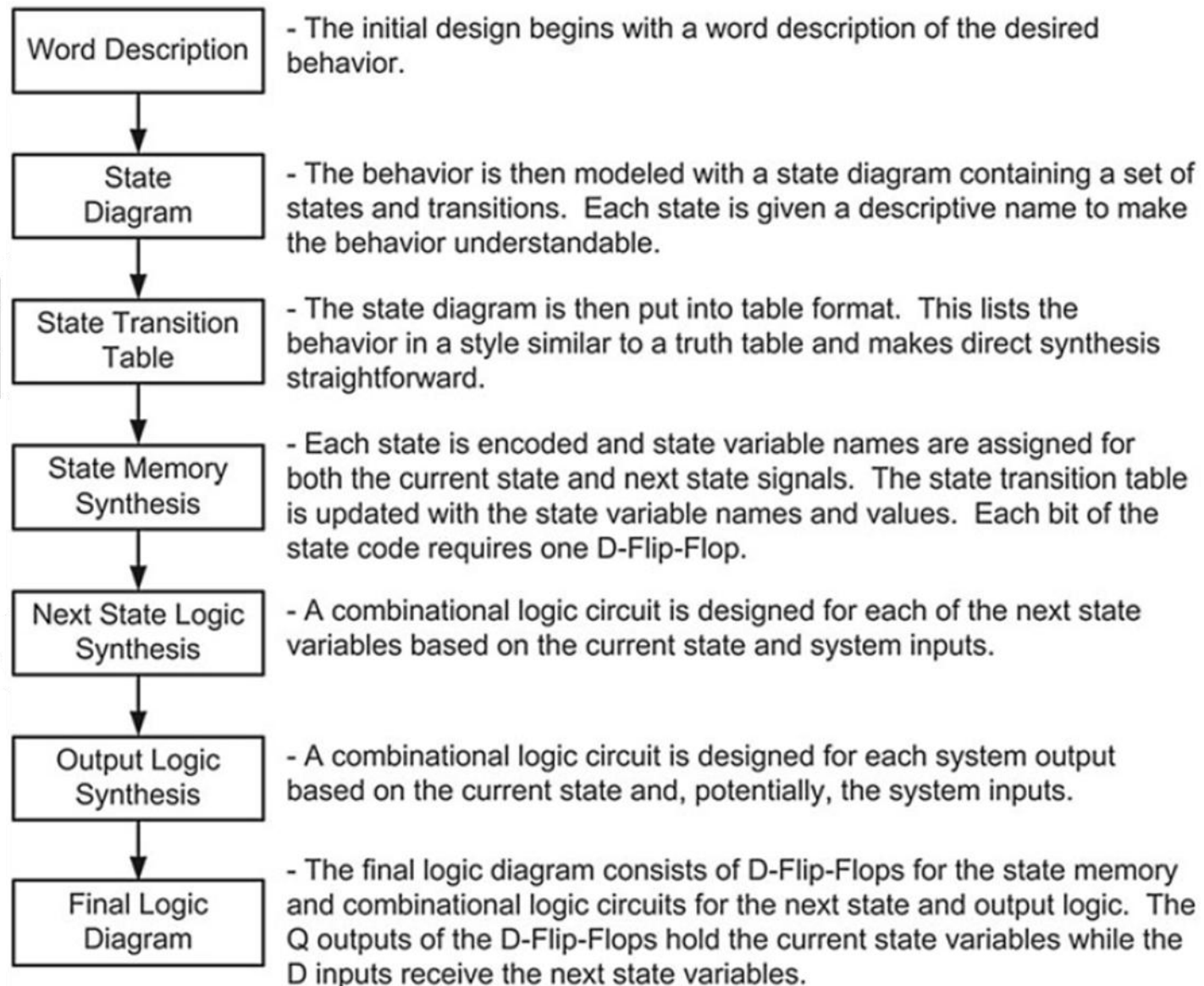
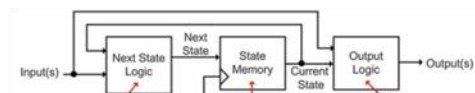
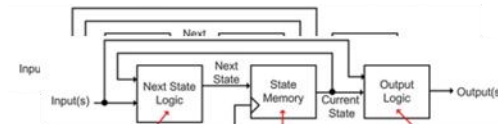
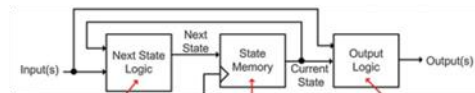


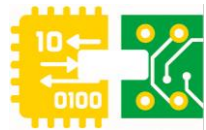
Current State		Input	Next State		Outputs	
	Q_cur	Press		Q_nxt	Open_CW	Close_CCW
w_closed	0	0	w_closed	0	0	0
w_closed	0	1	w_open	1	1	0
w_open	1	0	w_open	1	0	0
w_open	1	1	w_closed	0	0	1

Finite-State Machines: Synthesis



(Input)		(Outputs)		
Current State	Press	Next State	Open_CW	Close_CCW
w_closed	0	w_closed	0	0
w_closed	1	w_open	1	0
w_open	0	w_open	0	0
w_open	1	w_closed	0	1



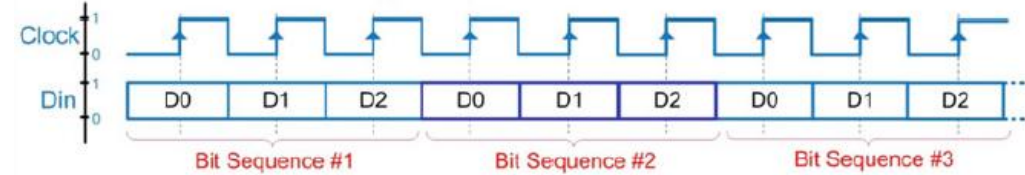
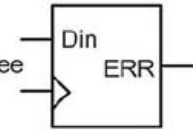


Finite-State Machines: Examples

- Serial bit sequence detector
 - Word description
 - State diagram and state transition table

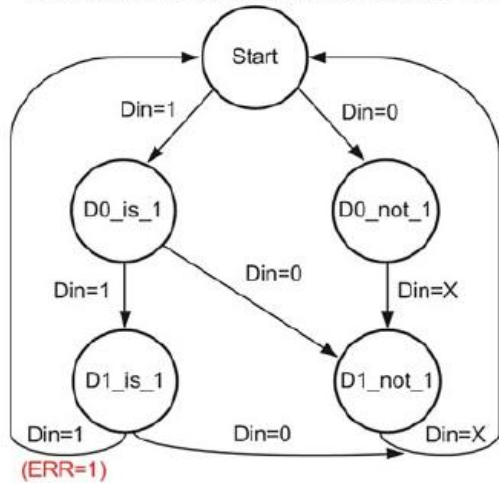
Word Description

We are going to design a circuit that will monitor an incoming serial bit stream. The information in the bit stream represents data in groups of three bits. The code "111" represents that an error has occurred in the transmitter. Our system needs to monitor the incoming bit stream and assert a signal called "ERR" if the sequence "111" is detected. At all other times and for all other incoming codes, ERR=0.



State Diagram & State Transition Table

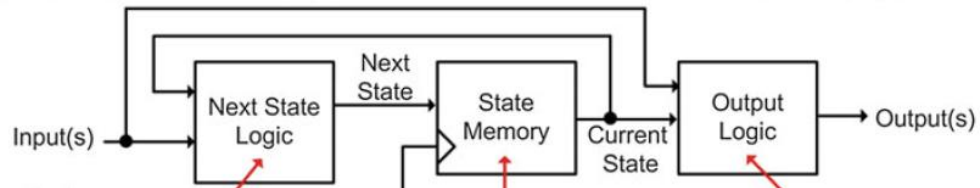
To implement this design, we need a machine that can keep track of the number of incoming bits. In this way, the machine will know once the three bits within a sequence have been received. The machine must also track if the sequence of incoming bits are 1's. In order to do this, let's create a sequence of states that will be traversed when Din=1. We also need a parallel sequence of states that will be traversed if an incoming bit is ever a 0. Each of these parallel paths must contain enough states to track that three bits of the sequence have been received before starting over and monitoring the next incoming sequence. The only time the output ERR will be asserted is when three 1's are received within one three bit data sequence. To simplify the state diagram, the output of ERR=1 is only listed once next to the corresponding transition in the diagram. It is assumed that at all other times, ERR=0.



Current State	(Input)		(Output)	
	Din	Next State	ERR	
Start	0	D0_not_1	0	
Start	1	D0_is_1	0	
D0_is_1	0	D1_not_1	0	
D0_is_1	1	D1_is_1	0	
D1_is_1	0	Start	0	
D1_is_1	1	Start	1	
D0_not_1	0	D1_not_1	0	
D0_not_1	1	D1_not_1	0	
D1_not_1	0	Start	0	
D1_not_1	1	Start	0	

Finite-State Machines: Examples

- Serial bit sequence detector
 - State encoding
 - Next state logic



State Encoding

Let's encode the states in binary in order to minimize the number of D-Flip-Flops. Encoding in Gray Code will not benefit this design since the state transitions are not linear. Since there are 5 unique states, we'll need 3 bits to encode all of the states. At this point, we also need to assign the state variable names. Let's call the three variables for the current state Q2_cur, Q1_cur, and Q0_cur. Let's call the three variables for the next state Q2_nxt, Q1_nxt, and Q0_nxt. After the state codes are assigned, we can update the state transition table.

State	Code
Start	= "000"
D0_is_1	= "001"
D1_is_1	= "010"
D0_not_1	= "011"
D1_not_1	= "100"

Current State				Input	Next State				Output
	Q2_cur	Q1_cur	Q0_cur	Din		Q2_nxt	Q1_nxt	Q0_nxt	ERR
Start	0	0	0	0	D0_not_1	0	1	1	0
Start	0	0	0	1	D0_is_1	0	0	1	0
D0_is_1	0	0	1	0	D1_not_1	1	0	0	0
D0_is_1	0	0	1	1	D1_is_1	0	1	0	0
D1_is_1	0	1	0	0	Start	0	0	0	0
D1_is_1	0	1	0	1	Start	0	0	0	1
D0_not_1	0	1	1	0	D1_not_1	1	0	0	0
D0_not_1	0	1	1	1	D1_not_1	1	0	0	0
D1_not_1	1	0	0	0	Start	0	0	0	0
D1_not_1	1	0	0	1	Start	0	0	0	0

Next State Logic

Q2_nxt

Q2_cur

Q1_cur

Q0_cur

Din

	00	01	11	10
00	0	0	X	0
01	0	0	X	0
11	0	1	X	X
10	1	1	X	X

Q1_nxt

Q2_cur

Q1_cur

Q0_cur

Din

	00	01	11	10
00	1	0	X	0
01	0	0	X	0
11	1	0	X	X
10	0	0	X	X

Q0_nxt

Q2_cur

Q1_cur

Q0_cur

Din

	00	01	11	10
00	1	0	X	0
01	1	0	X	0
11	0	0	X	X
10	0	0	X	X

$$Q0_nxt = (Q2_cur \cdot Q1_cur' \cdot Q0_cur')$$

$$Q1_nxt = (Q2_cur' \cdot Q1_cur' \cdot Q0_cur' \cdot Din') + (Q1_cur' \cdot Q0_cur \cdot Din)$$

$$Q2_nxt = (Q1_cur \cdot Q0_cur) + (Q0_cur \cdot Din')$$

Finite-State Machines: Examples

- Serial bit sequence detector
 - Output logic
 - Logic diagram

Q2_cur	Q1_cur	Q0_cur	Din	ERR
0	0	0	0	0
0	0	0	1	0
0	0	1	0	0
0	0	1	1	0
0	1	0	0	0
0	1	0	1	1
0	1	1	0	0
0	1	1	1	0
1	0	0	0	0
1	0	0	1	0

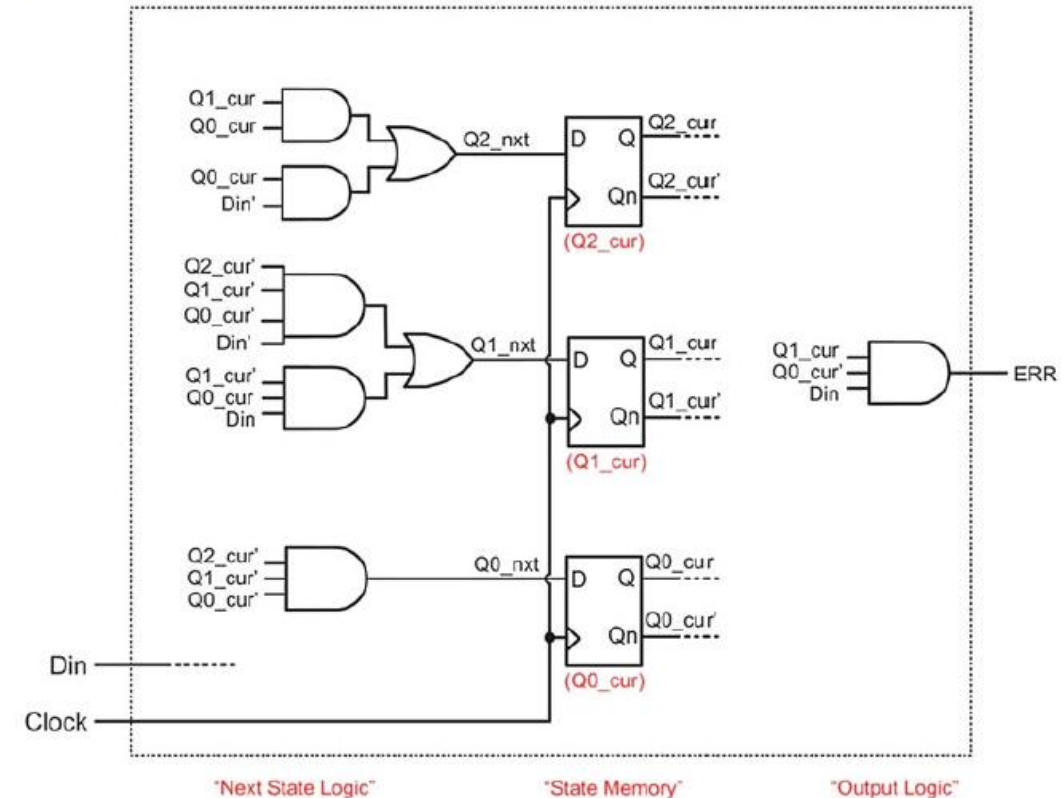
Output Logic

ERR

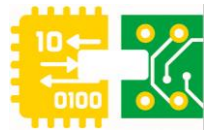
Q2_cur Q1_cur Q0_cur Din	00	01	11	10
00	0	0	X	0
01	0	1	X	0
11	0	0	X	X
10	0	0	X	X

→ $ERR = Q1_cur \cdot Q0_cur' \cdot Din$

Logic Diagram



Note that many of the wires are not drawn in to make the diagram readable. This is a common practice. Nodes with the same name are assumed to be connected regardless of whether a wire is drawn.

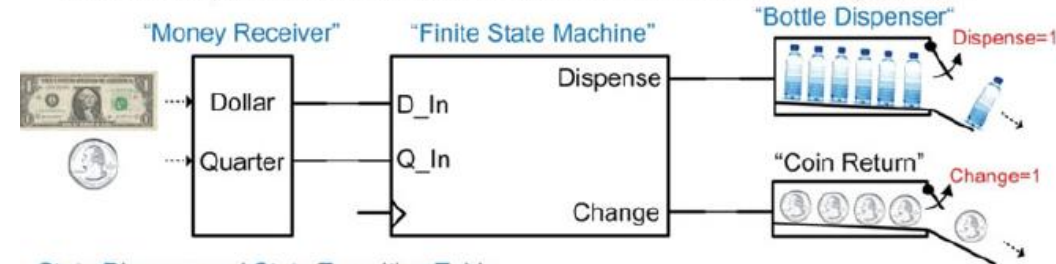


Finite-State Machines: Examples

- Vending Machine Controller
 - Word description
 - State diagram and state transition table

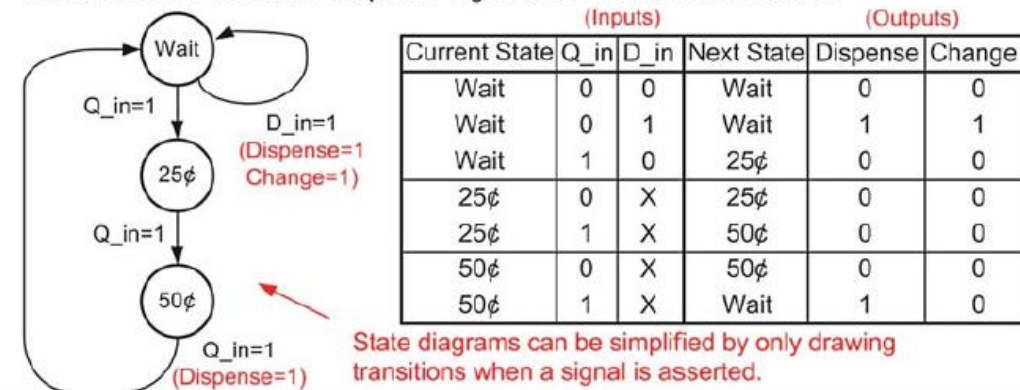
Word Description

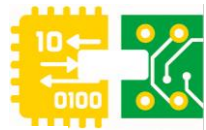
We are going to design a simple vending machine controller. The vending machine will sell bottles of water for 75¢. Customers can enter either a dollar bill or quarters. Once a sufficient amount of money is entered, the vending machine will dispense a bottle of water. If the user entered a dollar it will return one quarter in change. A "Money Receiver" detects when money has been entered. The receiver sends two logic signals to our circuit indicating whether a dollar bill or quarter was received. A "Bottle Dispenser" system holds the water bottles and will release one bottle when its input signal is asserted. A "Coin Return" system holds quarters for change and will release one quarter when its input signal is asserted. The money receiver will reject money if a dollar and quarter are entered simultaneously or if a dollar is entered once the user has started entering quarters.



State Diagram and State Transition Table

To implement this state machine, we will need an initial state that the machine will wait in until a customer enters money (Wait). If a dollar is entered, the machine will assert the "Dispense" signal to release a bottle of water and assert the "Change" signal to give one quarter in change. We do not need an additional state for the condition of when a dollar is entered because the machine will simply assert the output signals and return to the Wait state. When the customer pays with quarters, our machine needs to keep track of how many quarters have been received. We'll need two interim states that keep track of how many quarters have been entered (25¢ and 50¢). Once the third quarter has been entered our machine will assert the "Dispense" signal and return to the Wait state.





Finite-State Machines: Examples

- Vending Machine Controller
 - State encoding
 - Next state logic

State Encoding

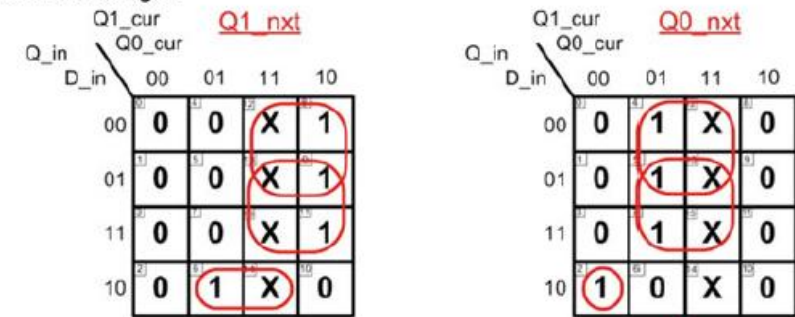
Let's encode the states in binary and name the current state variables Q1_cur and Q0_cur and the next state variables Q1_nxt and Q0_nxt. In this table we list out all possible values the current state and the inputs to make the table more complete.

State Code
 Wait = "00"
 25¢ = "01"
 50¢ = "10"

	Current State		Input		Next State		Outputs	
	Q1_cur	Q0_cur	Q_in	D_in	Q1_nxt	Q0_nxt	Dispense	Change
Wait	0	0	0	0	Wait	0	0	0
Wait	0	0	0	1	Wait	0	1	1
Wait	0	0	1	0	25¢	0	0	0
Wait	0	0	1	1	Wait	0	0	0
25¢	0	1	0	0	25¢	0	0	0
25¢	0	1	0	1	25¢	0	0	0
25¢	0	1	1	0	50¢	1	0	0
25¢	0	1	1	1	25¢	0	0	0
50¢	1	0	0	0	50¢	1	0	0
50¢	1	0	0	1	50¢	1	0	0
50¢	1	0	1	0	Wait	0	1	0
50¢	1	0	1	1	50¢	1	0	0

Next State Logic

The next state logic for this counter depends on both the current state variables and the system input Up. We can again take advantage of don't cares for the unused state code to minimize the logic.

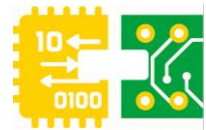


$$Q0_nxt = (Q0_cur \cdot Q_in') + (Q0_cur \cdot D_in) + (Q1_cur' \cdot Q0_cur' \cdot Q_in \cdot D_in')$$

$$Q1_nxt = (Q1_cur \cdot Q_in') + (Q1_cur \cdot D_in) + (Q0_cur \cdot Q_in \cdot D_in')$$

Finite-State Machines: Examples

- Vending Machine Controller
 - Output logic
 - Logic diagram



Output Logic

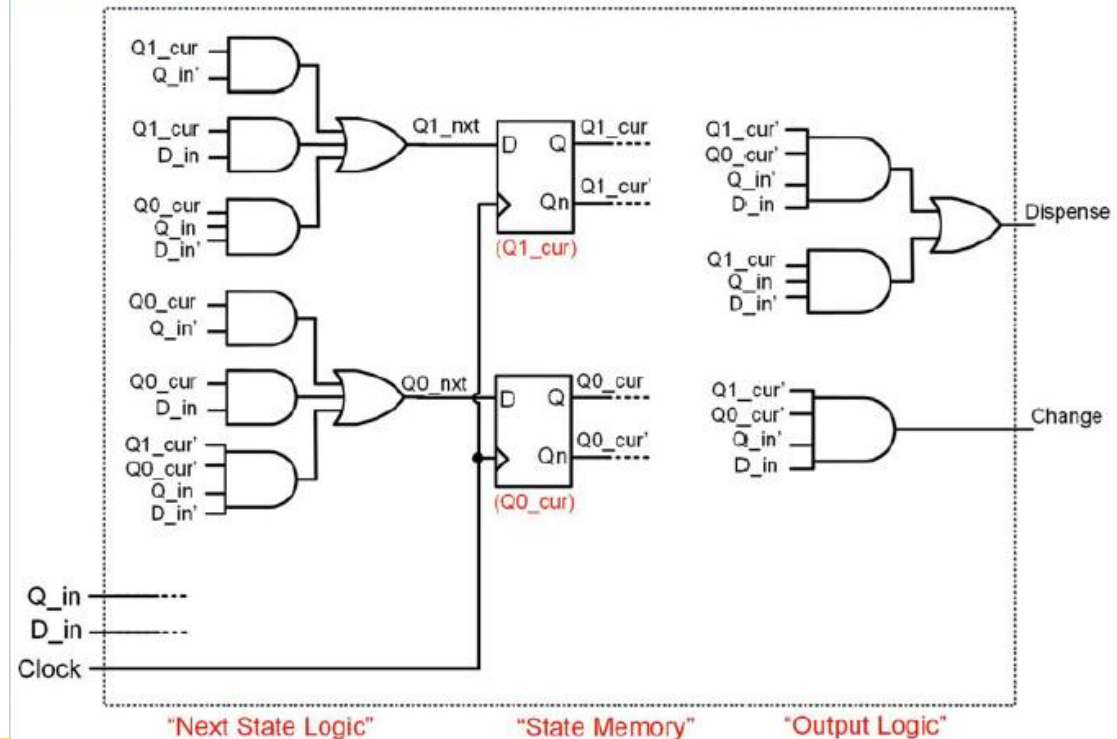
		Dispense			
Q1_cur	Q0_cur	00	01	11	10
00	01	0	0	X	0
01	11	1	0	X	0
11	10	0	0	X	0
10	00	0	0	X	1

		Change			
Q1_cur	Q0_cur	00	01	11	10
00	01	0	0	X	0
01	11	1	0	X	0
11	10	0	0	X	0
10	00	0	0	X	0

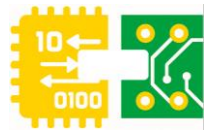
$$\text{Change} = (Q1_cur' \cdot Q0_cur' \cdot Q_in' \cdot D_in)$$

$$\text{Dispense} = (Q1_cur' \cdot Q0_cur' \cdot Q_in' \cdot D_in) + (Q1_cur \cdot Q_in \cdot D_in')$$

Logic Diagram

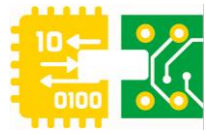


Finite-State Machines in VHDL



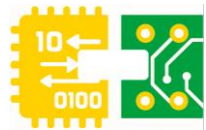
- Finite State Machine (FSM) is a synchronous sequential system specified by the following parameters:
 - set of states 
 - set of transitions (oriented) between these states 
 - set of transition conditions (logic expressions based on state machine inputs)  *Inputs / Outputs*
 - set of equations that specifies output values
 - Circuit transitions between states based on a triggering event, most commonly the edge of a clock
 - Circuit decision on the next state to transition to based on a history of past states and through the use of states and a predefined sequence of transitions
 - More intelligent outputs in comparison with a simple combinational logic circuit, whose outputs based only on the current values of the inputs

Finite-State Machines in VHDL



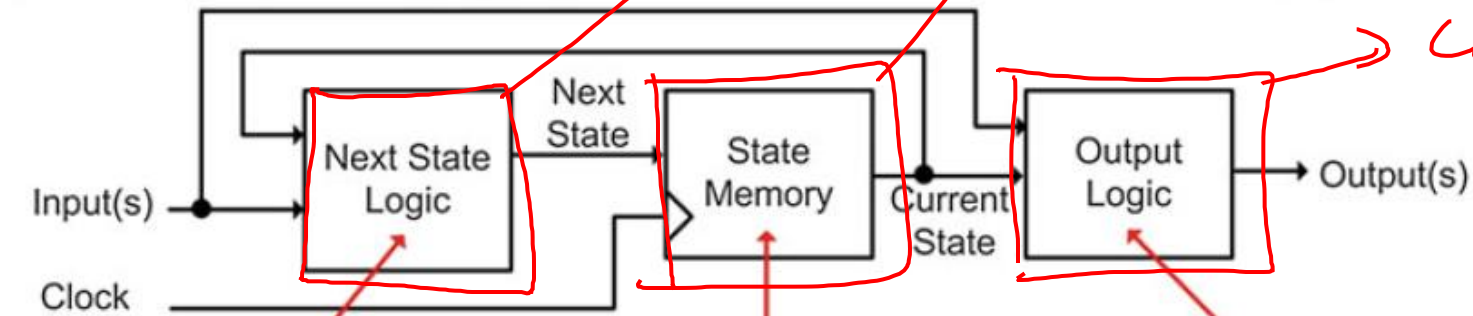
- State diagram: graphical representation of the functionality of an FSM
 - Graph, in which each **state** of the system denoted as a circle
 - **Transitions** moving to different states upon particular input conditions or remain in the same state
 - Two different types of output conditions for a state machine: Moore machine versus Mealy machine
 - Moore machine
 - Outputs dependence only of current state
 - Output generated for each specific state
 - Mealy machine
 - **Outputs** dependence of both current state and **system inputs**
 - Outputs written next to the state transitions corresponding to the appropriate output values

Finite-State Machines in VHDL



- Finite State Machine: Mealy and Moore

Mealy Machine – The output(s) depend on both the current state and system input(s).

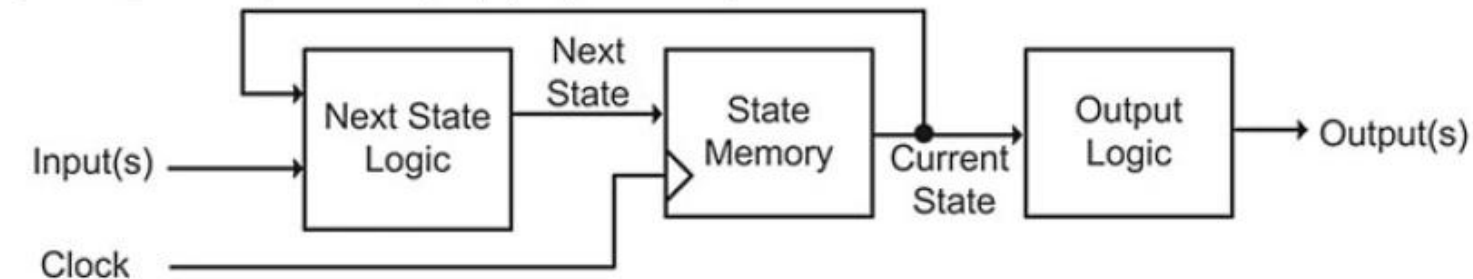


The next state logic creates the signal "next state" based on the current state and any system inputs. This block is implemented with combinational logic.

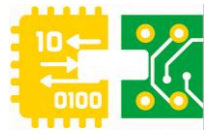
The state memory holds the current state. The current state is updated with "next state" on the rising edge of the clock. This block is implemented with D-Flip-Flops.

The output logic creates the system outputs. The output logic always depends on the current state of the machine and optionally (Mealy vs. Moore) the inputs of the system.

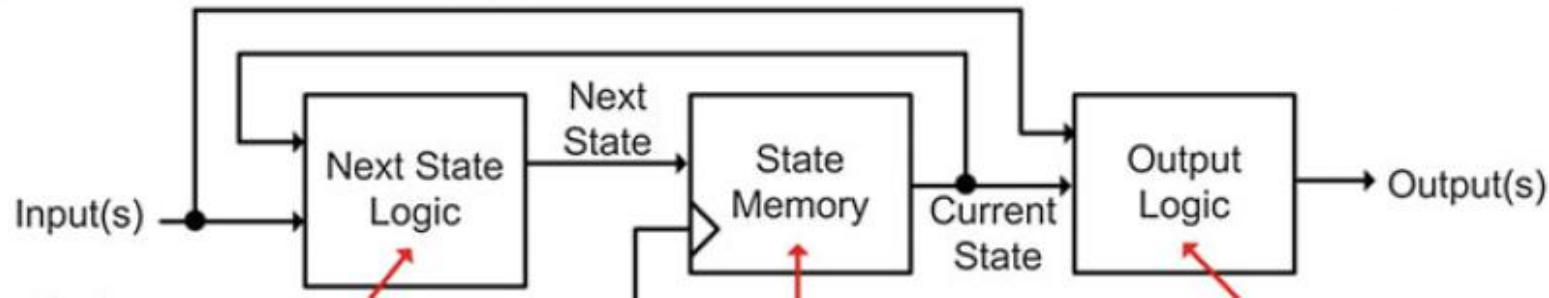
Moore Machine – The output(s) depends only on the current state.



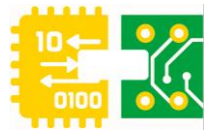
Finite-State Machines in VHDL



- Observations:

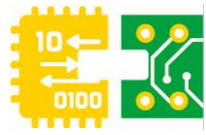


- How to realize states? New data type?
- How to realize transitions?
- Three functionalities = Three concurrent processes?



- **Question 1:** User defined data types: How to?
 - A type declaration defines the name of the type and the range of the type.
 - Type declaration are allowed in package declaration sections, entity declaration sections, architecture declaration sections, subprogram declaration section and process declaration sections
- Enumerated type is used to represent exactly the values required for a specific operation.
- All of the values of an enumerated type are user-defined.
- This values can be identifiers or single-character literals.

Finite-State Machines in VHDL



- User defined data type:

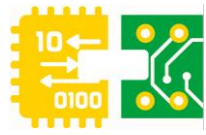
`type type_identifier is type_definition ;`

Handwritten annotations:
- "State" with an arrow pointing to "type_identifier"
- "(window-open, window-close)" with an arrow pointing to "type_definition"

- Example:

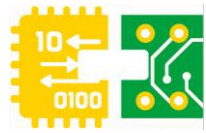
```
type small_integer is range 0 to 256;  
type my_vector_length is range 7 downto 0;  
type state is (start, state1, state2, end);
```

Finite-State Machines in VHDL



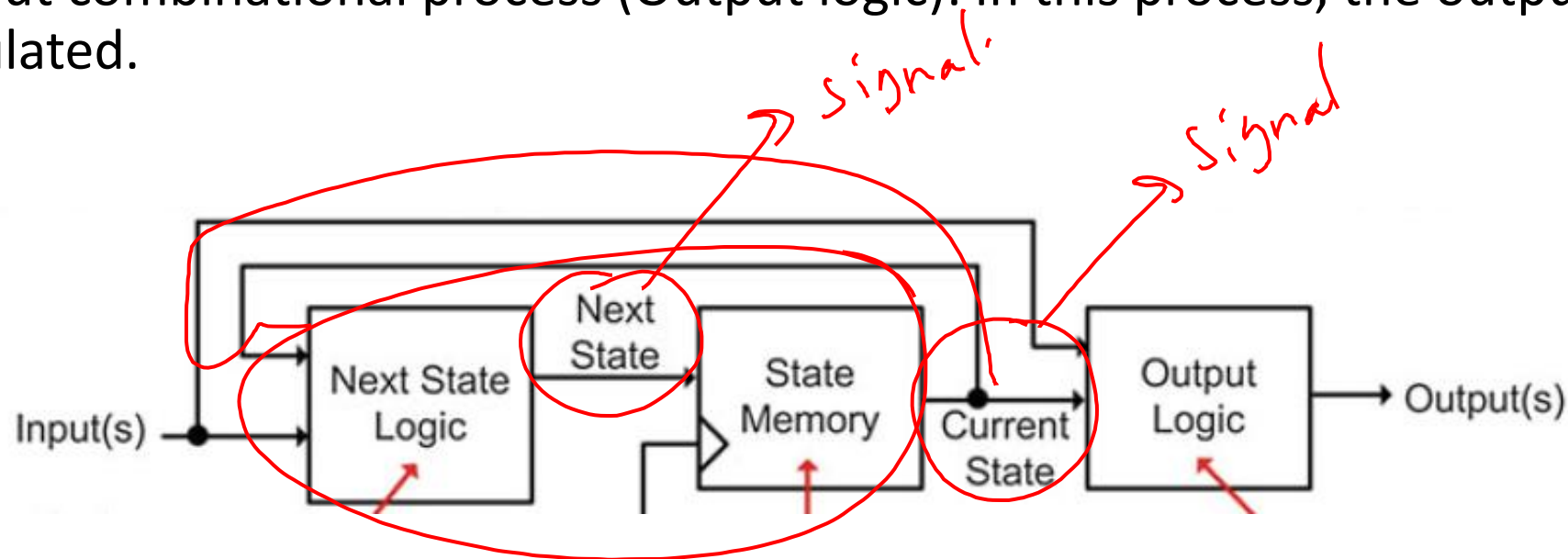
- **Question 2:** How many processes in VHDL for a FSM?
- Three-process FSM
 - The most logical method. Divide your model in many processes as possible (divide and conquer strategy). A process is a design unit, and it makes sense to make them as specialized as possible.
- Two-process FSM
 - Dividing the FSM implementation in two parts/processes.
 - One sequential, and the other combinational.
- One-process FSM
 - The most obvious method. Implement the whole functionality in one VHDL model. One fully synchronous process where the state, as well as the output being implemented in registers.

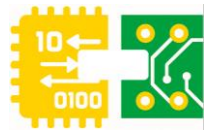
Finite-State Machines in VHDL



- Three-process method

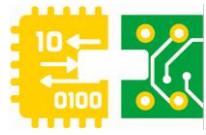
- Memory process (State memory): In this process, the next state is stored.
- Input combinational process (Next state logic): In this process, the next state is determined by the inputs.
- Output combinational process (Output logic): In this process, the outputs are calculated.



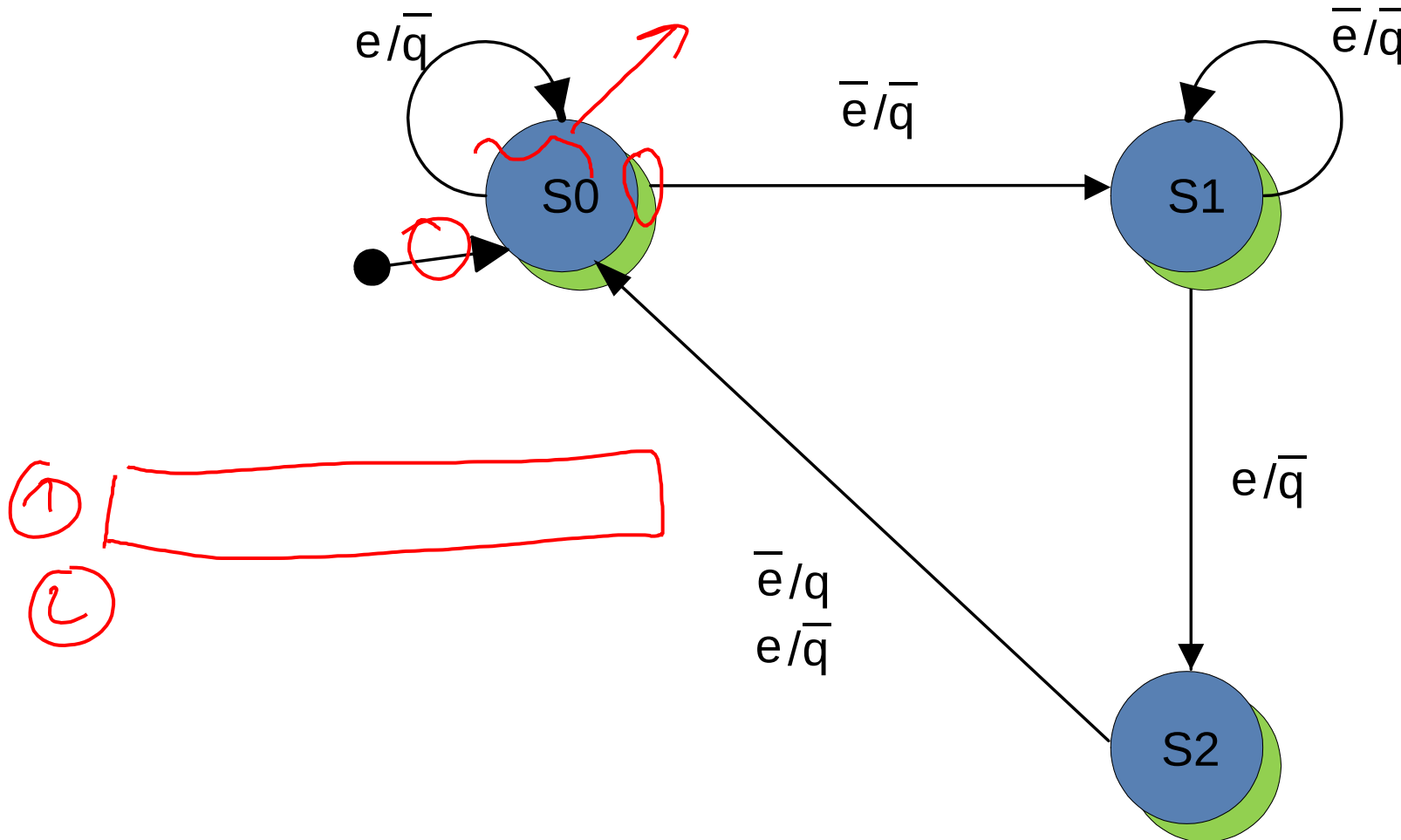


- Recommended guidelines:
 - Creation a new data type state introducing the different states of your machine.
 - The definition of a reset signal to initialize the machine to the initial state after power-on, and a clock signal.
 - The current state and the next state are represented each by a signal, the values of this **signal** are enumerated – defined by the user. Usually: `current_state` and `next_state`
 - Use a case statement to query the current state and determine the next state.
 - If all **cases** are not covered: use of "**when others**" as the last query based on a valid assignment.
 - To determine the outputs, use conditional assignments if needed.

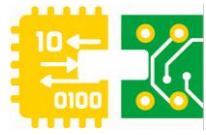
Finite-State Machines in VHDL



- Example:



Finite-State Machines in VHDL



- Entity and Architecture:

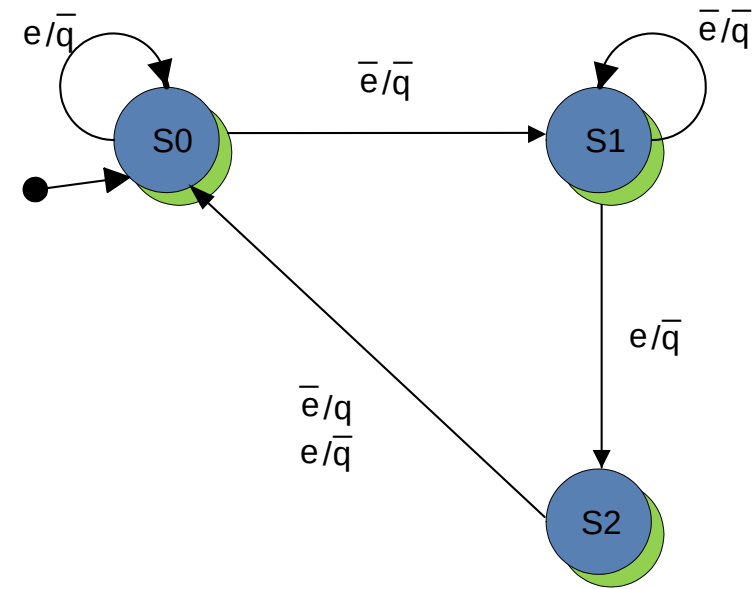
```
entity myFSM is
    port (rst, clk, e: in bit;
          q: out bit);
end entity myFSM;

architecture behavioral of myFSM is

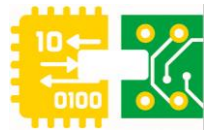
    type state is (S0, S1, S2);

    signal current_state, next_state: state;

begin
```



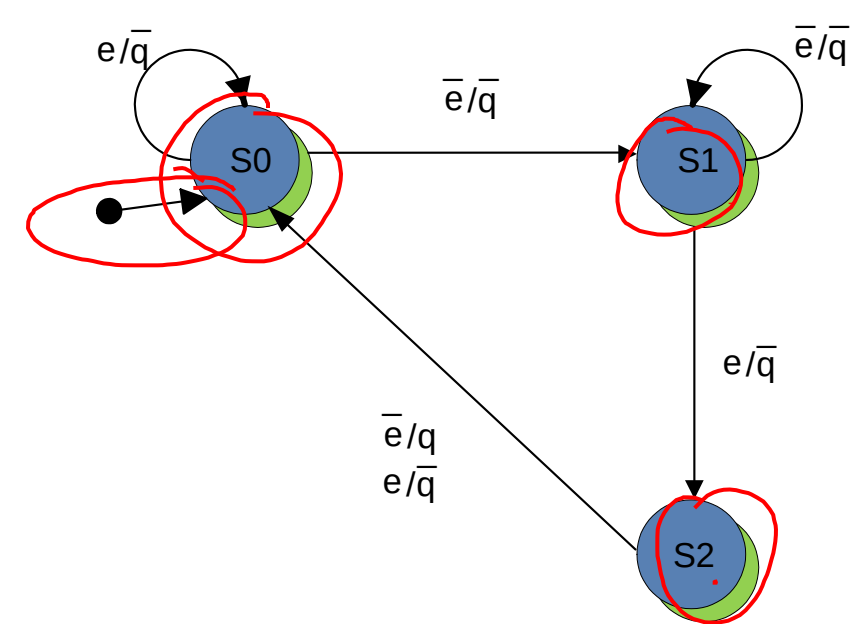
Finite-State Machines in VHDL



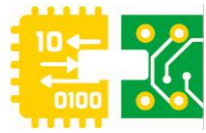
- Process 1: State memory

```
state_memory: process(clk, rst)
begin
    if rst = '1' then
        current_state <= S0;
    elsif (clk = '1' and clk'event) then
        current_state <= next_state;
    end if;
end process;
```

Handwritten notes: "all inputs" with an arrow pointing to the process name; "positive edge" with an arrow pointing to the clock condition; a checkmark at the bottom right.



Finite-State Machines in VHDL



- Process 2: Next state logic (1)

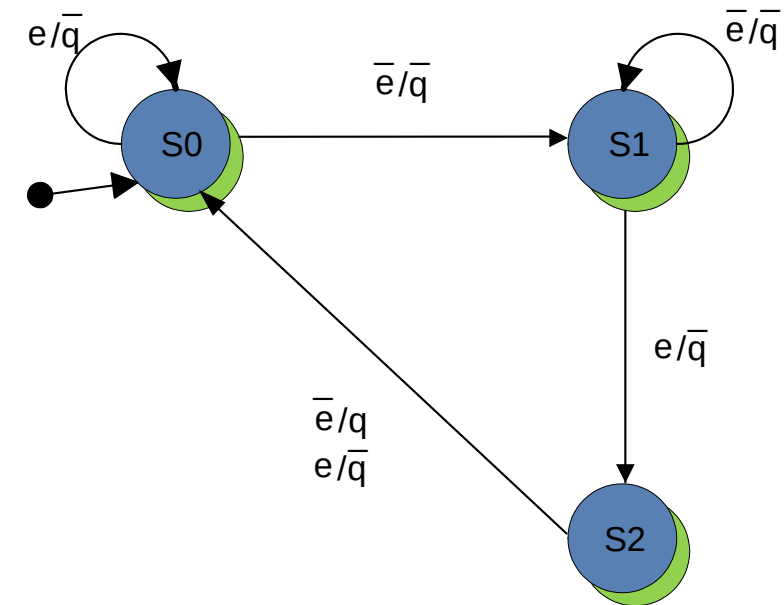
```
next_state_logic: process(current_state, e)
```

```
begin
```

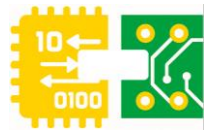
```
  case current_state is
```

```
    when S0 => if e = '1' then  
      next_state <= S0;  
    else next_state <= S1;  
    end if;
```

```
    when S1 => if e = '1' then  
      next_state <= S2;  
    else next_state <= S1;  
    end if;
```

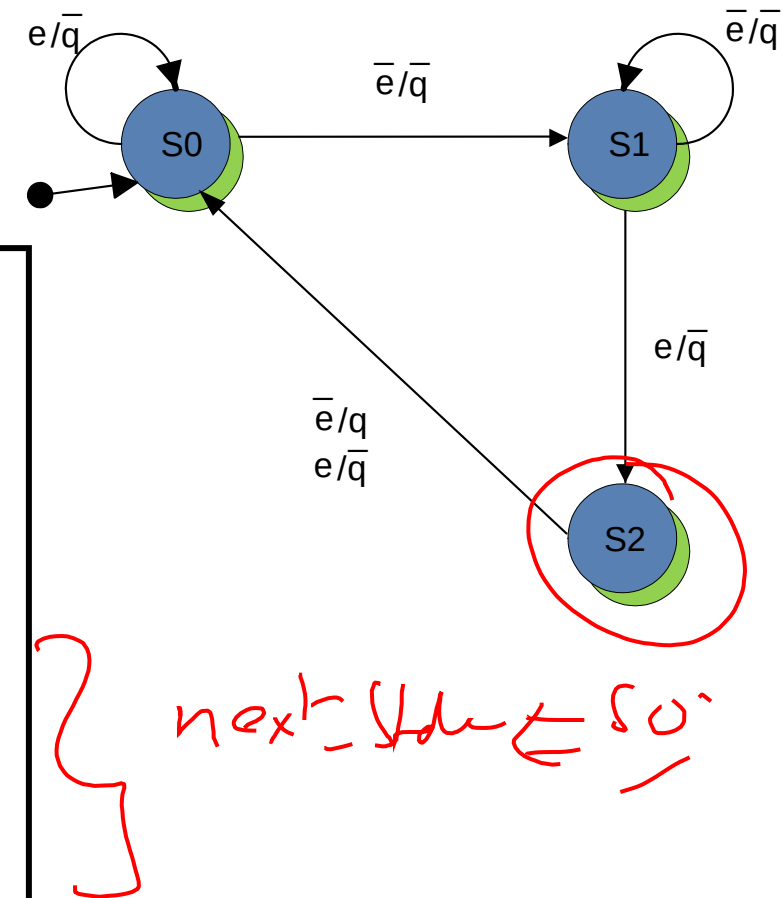


Finite-State Machines in VHDL

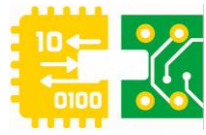


- Process 2: Next state logic (2)

```
next_state_logic: process(current_state, e)
begin
  case current_state is
    .
    .
    .
    when S2 => if e = '1' then
                  next_state <= S0; ✓
                else next_state <= S0; ✓
                end if;
  end case;
end process;
```

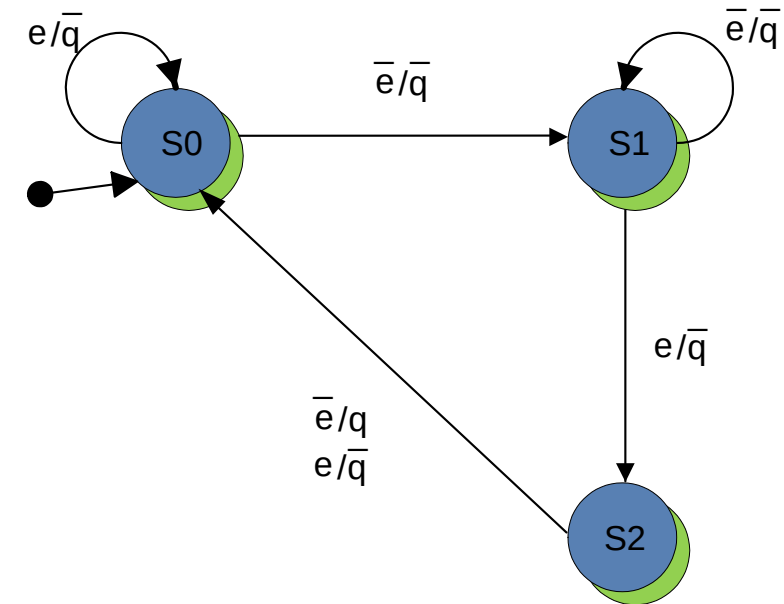


Finite-State Machines in VHDL

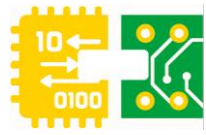


- Process 3: Output logic

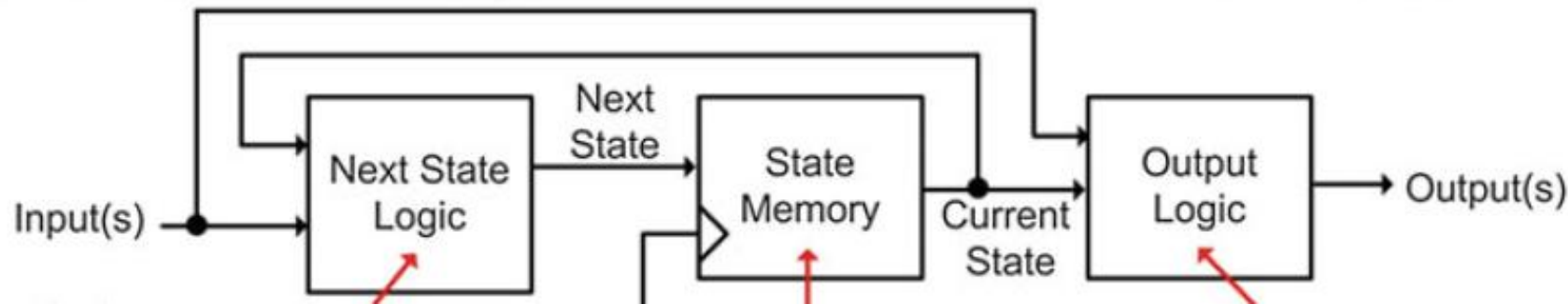
```
output_comb: process(current_state, e)
begin
  case current_state is
    when S2    =>    if e = '0' then
                     q <= '1';
                     else q <= '0';
                     end if;
    when others =>    q <= '0';
  end case;
end process;
end architecture;
```



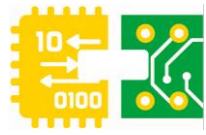
Finite-State Machines in VHDL



- Two-process method:
 - FSM code is divided into two processes. One synchronous, and the other combinational.
 - There are different ways that a two-process state machine can be implemented.

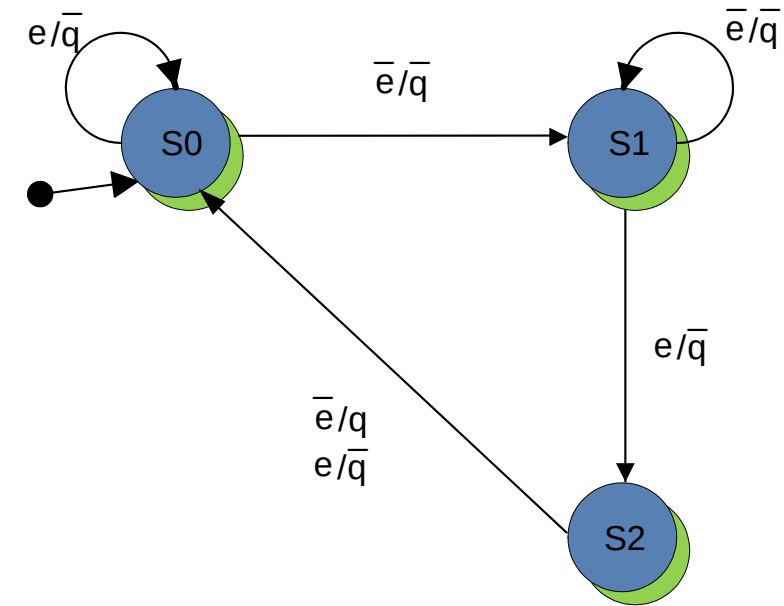


Finite-State Machines in VHDL

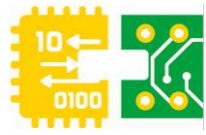


- Entity and Architecture:

```
entity myFSM is  
    port (rst, clk, e: in bit;  
          q: out bit);  
end entity myFSM;  
  
architecture behavioral of myFSM is  
  
    type state is (S0, S1, S2);  
  
    signal current_state, next_state: state;  
  
    begin
```

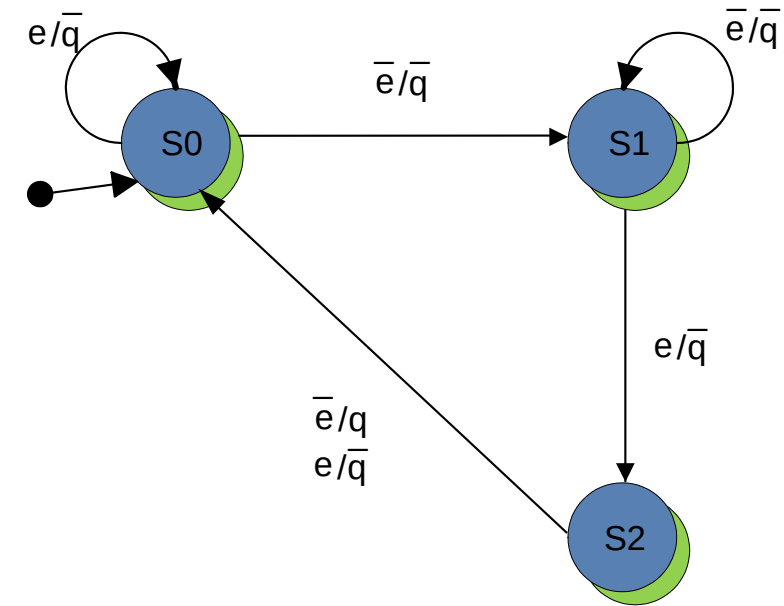


Finite-State Machines in VHDL

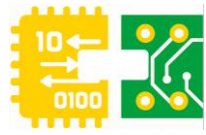


- Process 1: State memory

```
state_memory: process(clk, rst)
begin
  if rst = '1' then
    current_state <= S0;
  elsif (clk = '1' and clk'event) then
    current_state <= next_state;
  end if;
end process;
```



Finite-State Machines in VHDL

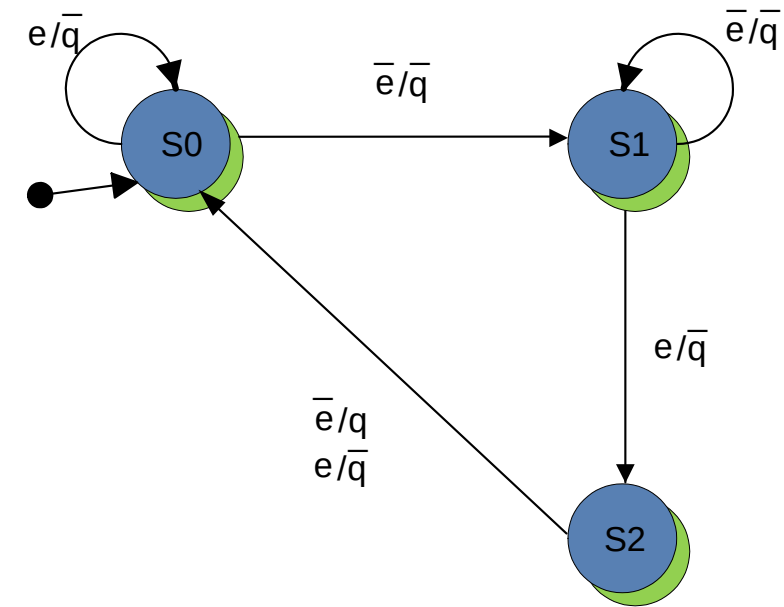


- Process 2: Combinational logic (1)

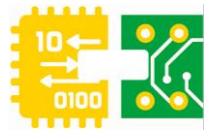
```
comb: process(current_state, e)

begin
  case current_state is
    when S0 => if e = '1' then
      next_state <= S0;
    else next_state <= S1;
    end if;
    q <= '0';

    when S1 => if e = '1' then
      next_state <= S2;
    else next_state <= S1;
    end if;
    q <= '0';
```



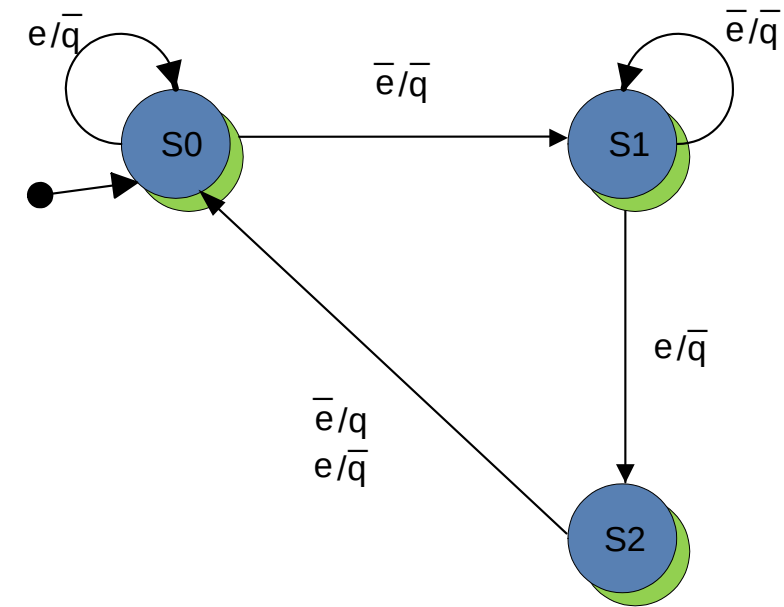
Finite-State Machines in VHDL



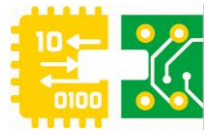
- Process 2: Combinational logic (2)

```
comb: process(current_state, e)

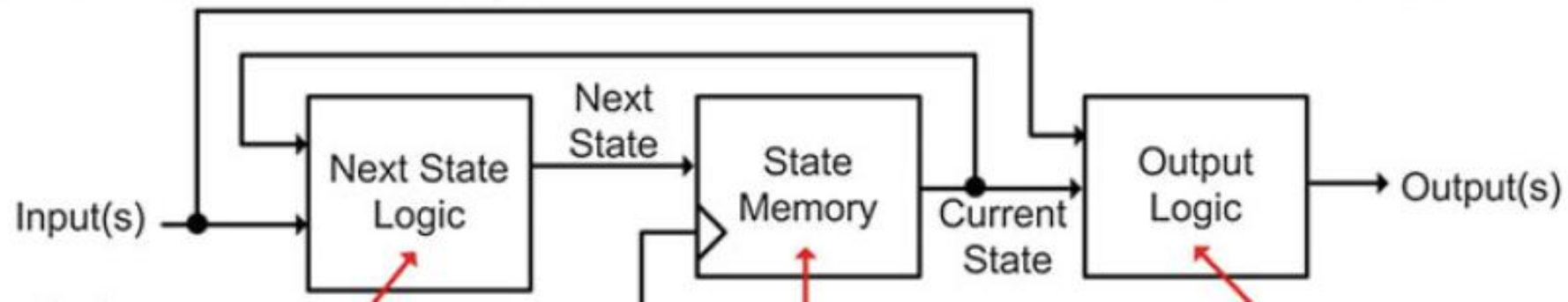
begin
  case current_state is
    .
    .
    .
    when S2 => if e = '1' then
                  next_state <= S0;
                  q <= '0';
                else next_state <= S0;
                  q <= '1';
                end if;
  end case;
end process;
end architecture;
```



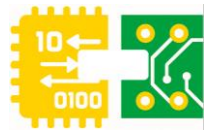
Finite-State Machines in VHDL



- One-process method:
 - The one-process method is the most obvious one.
 - It is usually implemented as a fully synchronous process.
 - This results in the state, as well as the output being implemented in registers.

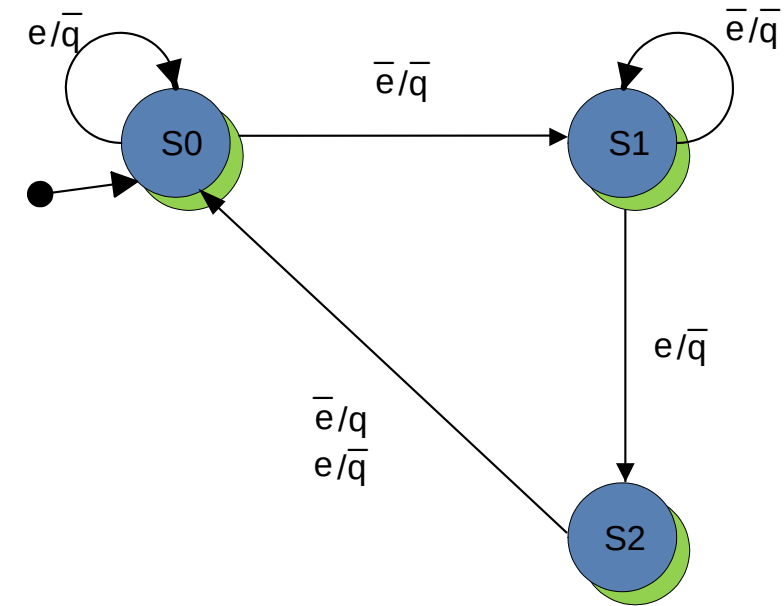


Finite-State Machines in VHDL

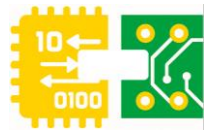


- Entity and Architecture:

```
entity myFSM is  
    port (rst, clk, e: in bit;  
          q: out bit);  
end entity myFSM;  
  
architecture behavioral of myFSM is  
  
    type state is (S0, S1, S2);  
  
    signal current_state: state;  
  
    begin
```

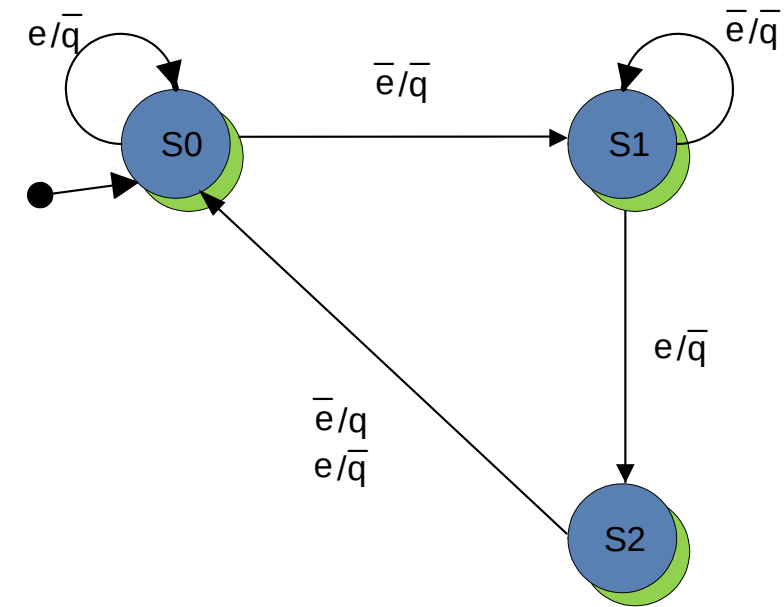


Finite-State Machines in VHDL

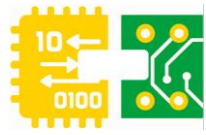


- Process: all functionalities (1)

```
fsm_all: process(clk, rst)
begin
  if rst = '1' then current_state <= S0;
  elsif (clk = '1' and clk'event) then
    case current_state is
      when S0 => if e = '1' then
                  current_state <= S0;
                else current_state <= S1;
                end if;
                  q <= '0';
      when S1 => if e = '1' then
                  current_state <= S2;
                else current_state <= S1;
                end if;
                  q <= '0';
```

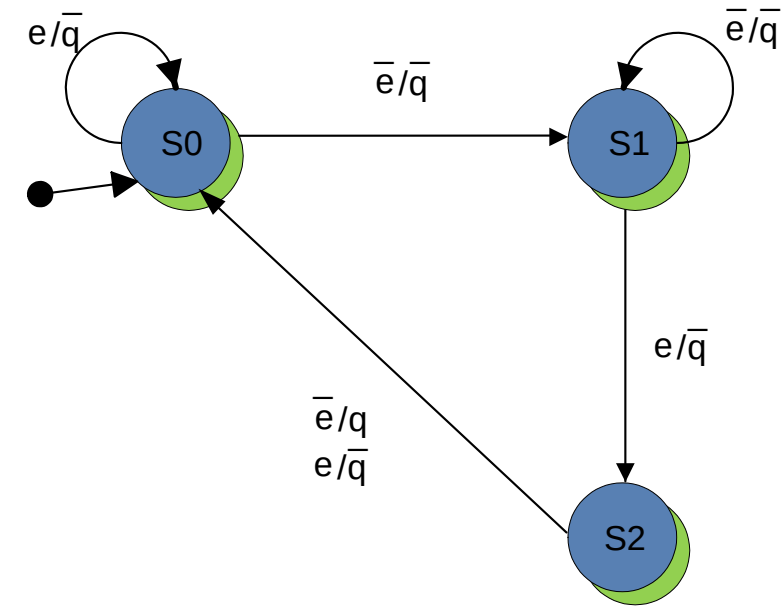


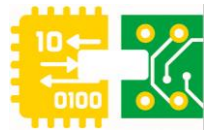
Finite-State Machines in VHDL



- Process: all functionalities (2)

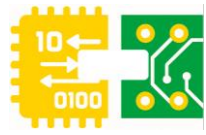
```
fsm_all: process(clk, rst)
begin
  if rst = '1' then current_state <= S0;
  elsif (clk = '1' and clk'event) then
    case current_state is
      .
      .
      when S2 => if e = '1' then
                    current_state <= S0;
                    q <= '0';
                  else current_state <= S0;
                    q <= '1';
                  end if;
    end case;
  end if;
end process;
end architecture;
```





- **Summary:** Which method should you use?
 - There is no right or wrong. None of the methods is absolutely the best for all cases. It always depends on different aspects:
 - Do you need a Moore machine or a Mealy machine?
 - Is the downstream logic that received the output signal synchronous or combinatorial?
 - It also depends on which style of coding you prefer.
 - The one-process FSM is probably the most common type. But multiple process FSMs are more used.
 - It depends on the background of the VHDL designer: designers that have a software background will probably choose the one-process method. Designers with electronics background will probably use a two- or three-process method.
 - More experienced designers are more likely to prefer the three-process method.

Finite-State Machines in VHDL



Summary: Guidelines:

1. Transform your idea to a text description
2. Define Inputs, Outputs and States
3. Describe the functionality using pseudo-code
4. Convert pseudo-code to FSM in state diagram
 - Define states and state transitions
 - Define datapath operations in each state.
5. Develop VHDL code to implement FSM using one of the 3 methods
6. Develop testbench for simulation and debugging
7. Test the implementation on FPGA boards



Thank you for your attention!

Prof. Dr.-Ing. Ali Hayek

University of Applied Sciences Hamm-Lippstadt (HSHL)

Tel.: **+ 49 (2381) 8789 - 924**

Room: **L4.2-E02-370**

E-mail: **ali.hayek@hshl.de**